

Assignment 2. ARP Spoofing DNS MitM with Scapy.

Author: Evgenii Dudkin

GitHub : <https://github.com/EvgeneDudkin/DAT505>

Introduction

This report presents the design and execution of ARP and DNS spoofing attacks using Scapy in a controlled virtual lab environment. The objective was to demonstrate how attackers can position themselves as a Man-in-the-Middle (MitM), intercept network traffic, and redirect victims to malicious destinations.

The assignment was divided into three tasks:

- Implementing an ARP spoofing script to poison ARP-tables of victim and gateway for establishing a MitM position.
- Capturing the traffic and development of a parser for analyzing this intercepted traffic to extract DNS queries, visited URLs, and other info.
- Developing a DNS spoofing script that targets specific domains and optionally forwards legitimate queries.

All experiments were performed using three isolated VMs: a Kali attacker, a victim machine, and a gateway providing Apache web server and DNS services. The project highlights both offensive techniques and practical mitigations, while considering the ethical implications of such attacks.

1 Lab Setup

The lab environment was created using Oracle VirtualBox (Figure 1), with three virtual machines connected via an Internal Network to isolate all traffic from external networks (Figure 2):

- Attacker VM: Kali Linux with Scapy, Python 3, Wireshark, and a basic HTTP server.
- Victim VM: Ubuntu, acting as a regular user with browser access.
- Gateway VM: Ubuntu, running Apache and dnsmasq to simulate a local DNS and web server.

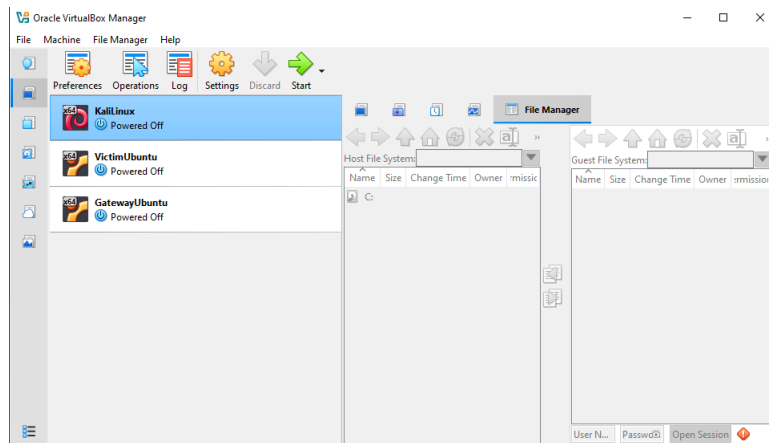


Figure 1: Lab Setup. VMs

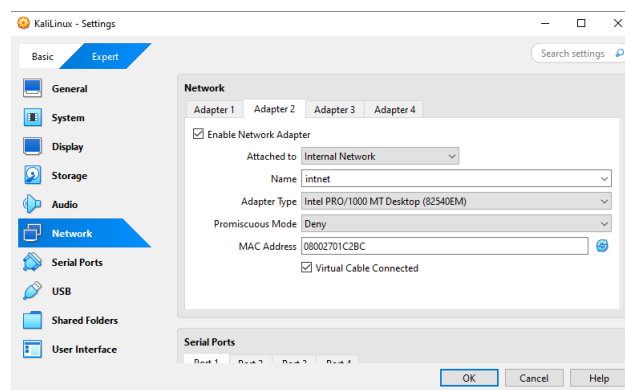


Figure 2: Lab Setup. Network settings

Each VM was configured with manual IP addressing to ensure predictable routing:

Machine	IP Address	Role
Gateway	192.168.2.1	DNS + Web Server
Victim	192.168.2.10	Target Host
Attacker	192.168.2.11	Kali (MitM)

The Victim and Attacker VMs use the Gateway IP (192.168.2.1) as their DNS server. Only the Attacker enables IP forwarding when needed (during ARP spoofing). All experiments were performed on this internal network to ensure safety and reproducibility.

2 Methodology

The experiment was divided into three main tasks. Each attack step was executed from the Kali attacker VM, while the Victim and Gateway performed their normal network roles. All scripts were written in Python using the Scapy library.

2.1 Task 1. ARP Spoofing

The script `arp_spoof.py` for poisoning the ARP caches of both victim and gateway was implemented as part of the task. Script was run on the Kali machine to send forged ARP replies to the victim and the gateway (Figure 3). This caused each to associate the attacker’s MAC address with the other’s IP address, enabling a full Man-in-the-Middle position.

The script accepted command-line arguments:

- `-t` for victim IP, `-s` for gateway IP, `-i` for interface
- `--ip-forward` (optional) to toggle traffic forwarding
- `-v` to enable verbose output

On exit (e.g. CTRL+C), the script restores the original ARP mappings.

ARP tables were manually checked on the Victim and Gateway to confirm poisoning.

```
evgeniidud@kali: ~/Desktop/ArpSpoof
(evgeniidud@kali)-[~/Desktop/ArpSpoof]
$ sudo python3 arp_spoof.py -t 192.168.2.10 -s 192.168.2.1 -i eth0 -v --ip-forward
[sudo] password for evgeniidud:
[*] IP forwarding enabled.
Begin emission

Finished sending 1 packets

Received 1 packets, got 1 answers, remaining 0 packets
/usr/lib/python3/dist-packages/scapy/sendrecv.py:479: SyntaxWarning: 'iface' has
no effect on L3 I/O send(). For multicast/link-local see https://scapy.readthed
ocs.io/en/latest/usage.html#multicast
warnings.warn(
WARNING: You should be providing the Ethernet destination MAC address when sendi
ng an is-at ARP.
Sent 1 packets.
[*] Spoofing 192.168.2.10 pretending to be 192.168.2.1
Begin emission

Finished sending 1 packets

Received 1 packets, got 1 answers, remaining 0 packets
```

Figure 3: ARP Spoofing. Attacker machine

2.2 Task 2. Traffic Capture & Analysis

During the ARP spoof, Wireshark was used on Kali machine to capture DNS and HTTP traffic. Two .pcap files were saved (dns.pcapng, http.pcapng).

The script scapy_pcap_parser.py was implemented to extract: DNS queries, visited HTTP hosts, top IP talkers (by packet count), protocol usage (TCP, UDP, etc.). The parser takes the input and output files as arguments. It processes .pcap file and its output is saved to a structured .txt file for analysis.

2.3 Task 3. DNS Spoofing

For this task script dns_spoof.py was implemented to intercept victim DNS queries and reply with attacker-controlled IPs for selected domains.

It works as follows:

- Intercepted DNS requests from the victim
- If the domain was listed in a whitelist file (spoof_list.txt), it spoofed the response with Kali's IP
- For other domains, it forwarded the query to the real DNS server (gateway), received the reply, and relayed it back to the victim

To prevent real DNS replies from overriding spoofed ones, iptables was used to drop forwarded responses from port 53. A simple HTTP server was launched on Kali (python3 -m http.server 80) to serve a fake page. When the victim opened a spoofed domain (e.g. example.com), the browser loaded the attacker's web content.

3 Results

This section presents the evidence confirming that each attack was successful. Screenshots, packet captures, and logs were collected during the attack phases. All tools were executed on the Kali attacker VM, with observations made on both the Victim and Gateway.

3.1 ARP Spoofing Verification

After launching the ARP spoofing script on Kali, the following was observed:

- The Victim's ARP table showed the gateway IP (192.168.2.1) mapped to Kali's MAC address

- The Gateway's ARP table showed the victim IP (192.168.2.10) mapped to Kali's MAC
- Traffic between the two hosts passed through the attacker (Figure 8)

Figure 4 shows the Victim's ARP table before attack, while Figure 5 presents its state during attack. Figure 6 and Figure 7 show the Gateway's ARP table before and during attack respectively.

```
victimubuntu@victimubuntu:~$ ip neigh
192.168.2.11 dev enp0s3 lladdr 08:00:27:01:c2:bc REACHABLE
192.168.2.1 dev enp0s3 lladdr 08:00:27:5d:ff:4f REACHABLE
```

Figure 4: Victim's ARP table before attack

```
victimubuntu@victimubuntu:~$ ip neigh
192.168.2.11 dev enp0s3 lladdr 08:00:27:01:c2:bc STALE
192.168.2.1 dev enp0s3 lladdr 08:00:27:01:c2:bc REACHABLE
```

Figure 5: Victim's ARP table during attack

```
gatewayubuntu@gatewayubuntu:~$ arp -n
Address          HWtype  HWaddress      Flags Mask    Iface
192.168.2.11     ether   08:00:27:01:c2:bc C              enp0s3
192.168.2.10     ether   08:00:27:01:72:e4 C              enp0s3
```

Figure 6: Gateway's ARP table before attack

Since the script includes a graceful restore on exit, all ARP tables on the victim and gateway automatically returned to their original state after the attack was stopped, ensuring the network returned to normal conditions.

3.2 Traffic Capture and Analysis

Network traffic was captured using Wireshark on the Kali VM during the attack, and analyzed using a custom Scapy-based parser to extract DNS queries, HTTP hosts, protocol usage, and top IP talkers. The extracted data matched expected traffic patterns and confirmed the attacker's position in the communication flow. Figure 9 shows an example of the parser output on a .pcap file containing captured HTTP traffic between the victim and the gateway.

```
gatewayubuntu@gatewayubuntu:~$ arp -n
```

Address	HWtype	HWaddress	Flags	Mask	Iface
192.168.2.11	ether	08:00:27:01:c2:bc	C		enp0s3
192.168.2.10	ether	08:00:27:01:c2:bc	C		enp0s3

Figure 7: Gateway’s ARP table during attack

The image shows a Wireshark packet capture window titled 'arp_spoof_attack.pcapng'. The filter bar shows 'http && ip.addr == 192.168.2.10'. The packet list shows four packets, all of which are HTTP GET requests from 192.168.2.10 to 192.168.2.11. The packet details pane shows the first packet, a GET request for 'http://192.168.2.11/'.

No.	Time	Source	Destination	Protocol	Length	Info
1589	74.409627258	192.168.2.10	192.168.2.11	HTTP	497	GET / HTTP/1.1
1597	74.480551230	192.168.2.1	192.168.2.10	HTTP	3526	HTTP/1.1 200 OK (text/html)
1613	75.802672575	192.168.2.10	192.168.2.11	HTTP	497	GET / HTTP/1.1
1616	75.803698835	192.168.2.1	192.168.2.10	HTTP	3525	HTTP/1.1 200 OK (text/html)

Figure 8: Wireshark view from Kali showing intercepted traffic

3.3 DNS Spoofing Verification

As a result of the attack, the Kali machine responded to DNS queries for example.com with its own IP address (192.168.2.11), since the domain was included in the spoofing whitelist.

Figure 10 shows multiple spoofed DNS responses captured in Wireshark, sent from Kali to the victim. Following this, when the victim accessed example.com in the browser, the request was redirected to the attacker’s web server instead of the legitimate Apache page.

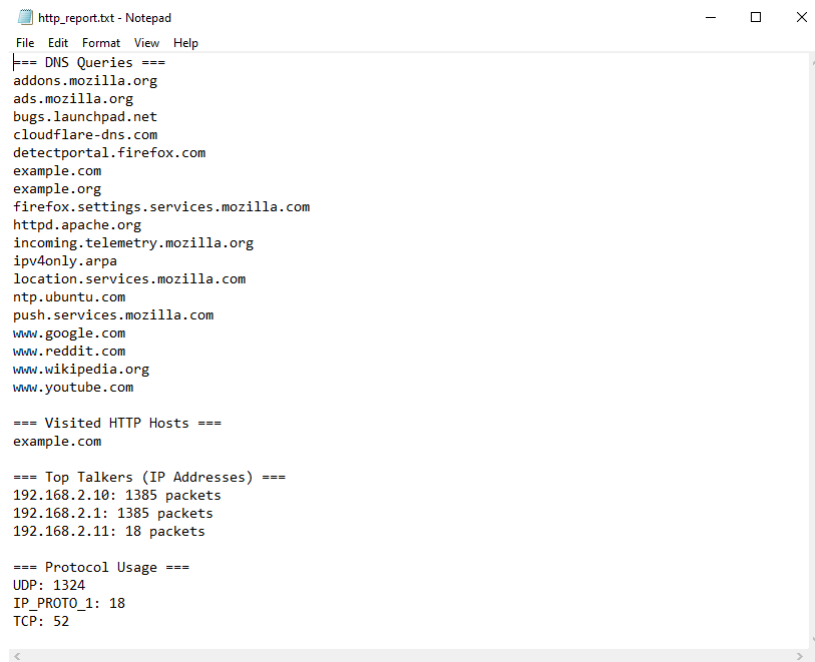
Figure 11 shows the spoofed site being successfully loaded in the victim’s browser.

4 Mitigations

ARP spoofing can be mitigated using static ARP entries or monitoring tools like ARPwatch. DNS spoofing is preventable with DNSSEC, which authenticates DNS responses. Using HTTPS also protects users by preventing content tampering even if DNS is compromised.

5 Ethics section

All attacks in this lab were performed in a fully isolated virtual environment, with no impact on external systems. The purpose of this exercise was to better understand network vulnerabilities and try to exploit them for learning purposes.



```
http_report.txt - Notepad
File Edit Format View Help
=== DNS Queries ===
addons.mozilla.org
ads.mozilla.org
bugs.launchpad.net
cloudflare-dns.com
detectportal.firefox.com
example.com
example.org
firefox.settings.services.mozilla.com
httpd.apache.org
incoming.telemetry.mozilla.org
ipv4only.arpa
location.services.mozilla.com
ntp.ubuntu.com
push.services.mozilla.com
www.google.com
www.reddit.com
www.wikipedia.org
www.youtube.com

=== Visited HTTP Hosts ===
example.com

=== Top Talkers (IP Addresses) ===
192.168.2.10: 1385 packets
192.168.2.1: 1385 packets
192.168.2.11: 18 packets

=== Protocol Usage ===
UDP: 1324
IP_PROTO_1: 18
TCP: 52
```

Figure 9: Parser output

Techniques like ARP and DNS spoofing are powerful but must only be used in controlled, ethical contexts for testing or education.

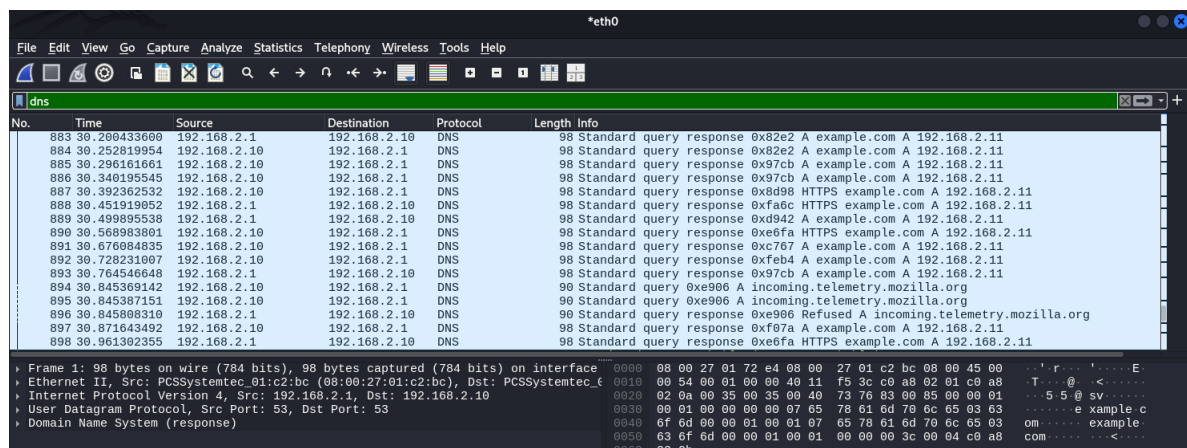


Figure 10: Wireshark view from Kali showing DNS spoofing messages

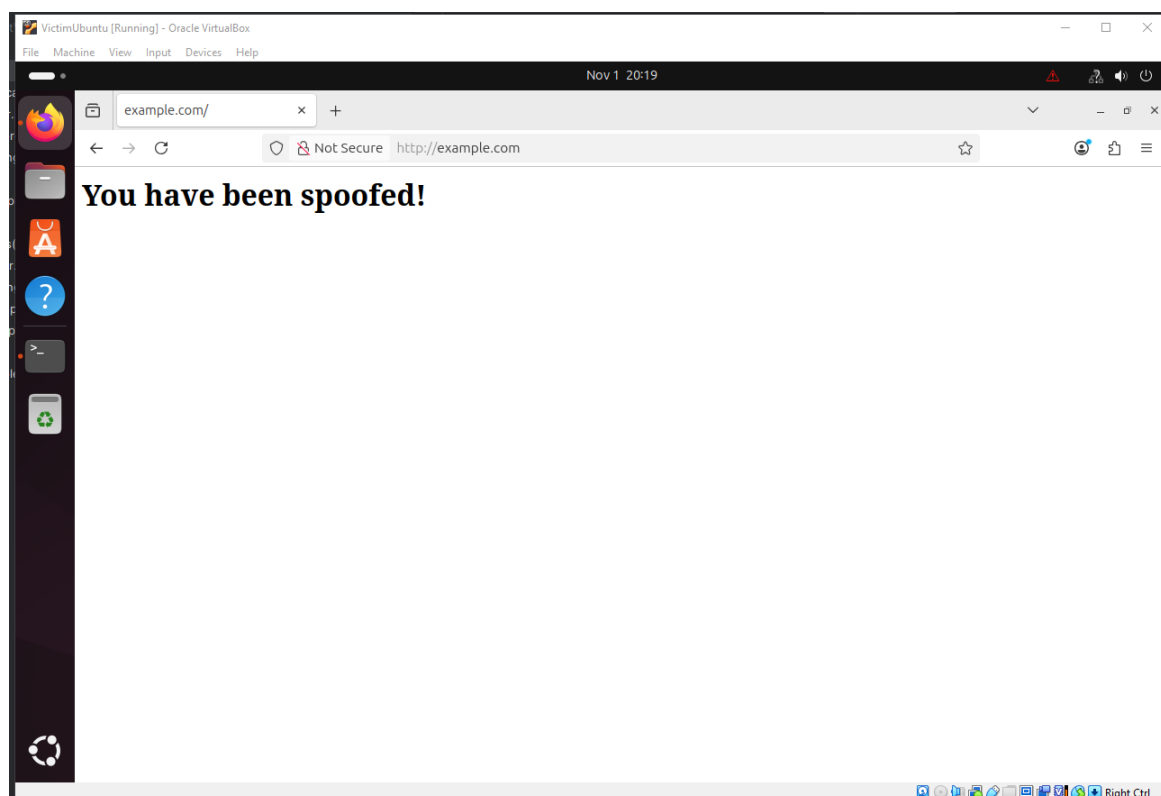


Figure 11: Victim's browser loading the spoofed web page after being redirected to the attacker's IP for example.com